

0.1 Thiết kế kiến trúc

0.1.1 Áp dụng kiến trúc MVC trong ứng dụng quản lý tài sản

ứng dụng quản lý tài sản được xây dựng dựa trên kiến trúc phần mềm MVC (Model – View – Controller), trong đó mỗi thành phần đảm nhận một vai trò riêng biệt nhằm đảm bảo tính tách rời, dễ bảo trì và mở rộng hệ thống. Việc áp dụng kiến trúc MVC giúp hệ thống quản lý tài sản hoạt động ổn định, rõ ràng về luồng xử lý và dễ dàng tích hợp thêm các chức năng mới.

a, Thành phần Model

Thành phần Model trong ứng dụng quản lý tài sản chịu trách nhiệm quản lý dữ liệu, ánh xạ với cơ sở dữ liệu và triển khai các nghiệp vụ liên quan đến quản lý tài sản. Mỗi Model thường tương ứng với một bảng dữ liệu và định nghĩa các mối quan hệ giữa các thực thể trong hệ thống.

Một số Model tiêu biểu trong ứng dụng quản lý tài sản bao gồm:

- **Asset:** Quản lý thông tin tài sản như mã tài sản, trạng thái, ngày mua, người đang sử dụng và lịch sử cấp phát.
- **User:** Lưu trữ và quản lý thông tin người dùng, bao gồm quyền truy cập và vai trò trong hệ thống.
- **Category:** Quản lý các nhóm phân loại tài sản như máy tính, thiết bị mạng, phần mềm.
- **Manufacturer:** Lưu thông tin nhà sản xuất của tài sản, phục vụ việc thống kê và quản lý nguồn gốc.
- **Location:** Quản lý phòng học lưu trữ hoặc sử dụng tài sản, hỗ trợ việc theo dõi tài sản theo địa điểm.

Các Model này giúp đảm bảo dữ liệu trong hệ thống được tổ chức chặt chẽ và nhất quán, đồng thời hỗ trợ các nghiệp vụ quản lý tài sản một cách hiệu quả.

b, Thành phần Controller

Controller trong ứng dụng quản lý tài sản đóng vai trò tiếp nhận yêu cầu từ người dùng, xử lý logic nghiệp vụ và điều phối dữ liệu giữa Model và View. Mỗi Controller thường chịu trách nhiệm cho một nhóm chức năng cụ thể trong hệ thống.

Một số Controller tiêu biểu bao gồm:

- **AssetsController:** Xử lý các chức năng liên quan đến tạo mới, cập nhật, xóa và hiển thị thông tin tài sản.

- **UsersController:** Quản lý người dùng, phân quyền và xác thực truy cập.
- **CategoriesController:** Quản lý danh mục và phân loại tài sản.
- **LocationsController:** Xử lý các nghiệp vụ liên quan đến vị trí sử dụng hoặc lưu trữ tài sản.
- **CheckinCheckoutController:** Thực hiện các nghiệp vụ cấp phát, thu hồi và theo dõi lịch sử sử dụng tài sản.

Việc phân chia Controller theo chức năng giúp mã nguồn rõ ràng, dễ bảo trì và thuận tiện khi mở rộng thêm các nghiệp vụ mới cho hệ thống.

c, Thành phần View

Thành phần View trong ứng dụng quản lý tài sản chịu trách nhiệm hiển thị giao diện và dữ liệu cho người dùng. Các View được xây dựng bằng Blade Template Engine của Laravel, giúp tách biệt hoàn toàn giao diện khỏi logic xử lý nghiệp vụ.

Một số View tiêu biểu trong hệ thống bao gồm:

- **Danh sách tài sản:** Hiển thị toàn bộ tài sản cùng các thông tin cơ bản như trạng thái, loại tài sản và người sử dụng.
- **Trang chi tiết tài sản:** Hiển thị thông tin chi tiết của một tài sản, bao gồm lịch sử cấp phát và tình trạng hiện tại.
- **Danh sách người dùng:** Hiển thị thông tin người dùng và quyền truy cập trong hệ thống.
- **Trang quản lý danh mục:** Cho phép xem và chỉnh sửa các loại tài sản.
- **Trang quản lý vị trí:** Hiển thị và quản lý các vị trí lưu trữ hoặc sử dụng tài sản.

Các View chỉ tập trung vào việc hiển thị dữ liệu, không xử lý logic nghiệp vụ phức tạp, giúp giao diện dễ chỉnh sửa và nâng cấp trong tương lai.

0.1.2 Thiết kế tổng quan

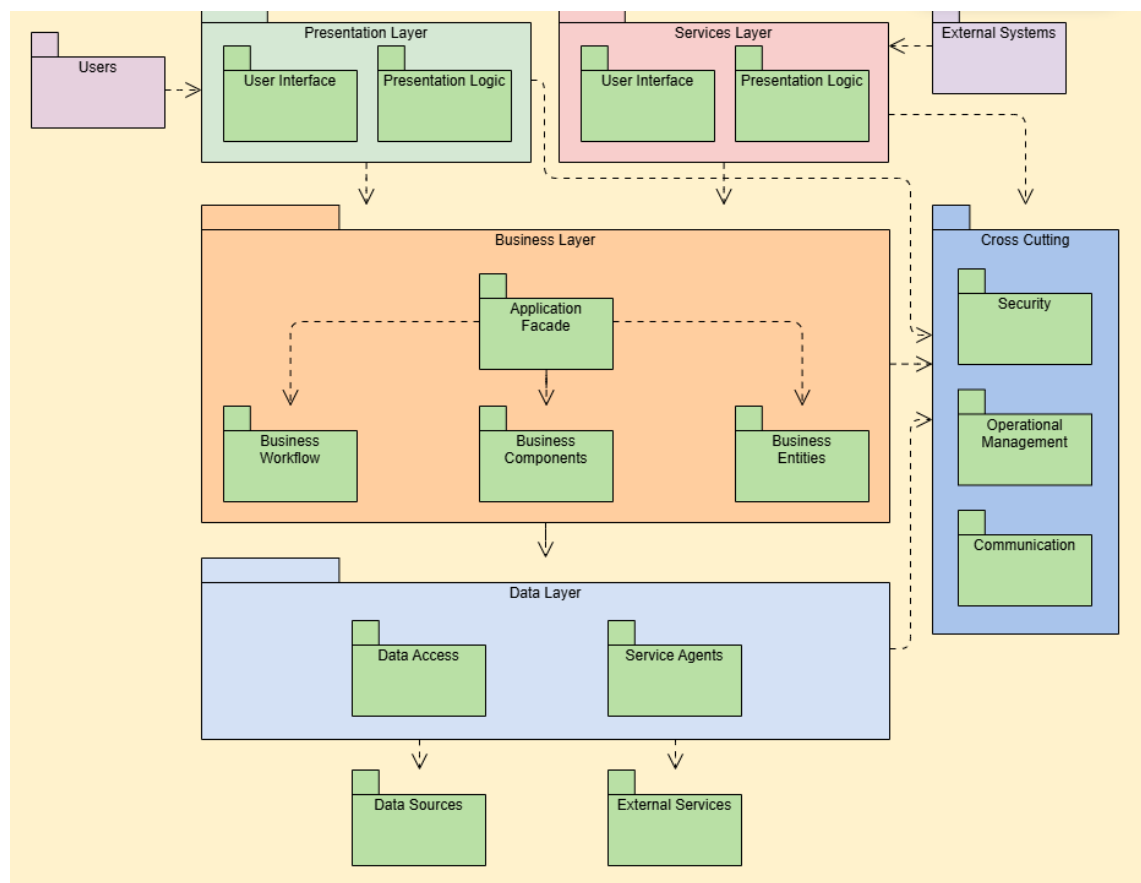
Trong giai đoạn thiết kế tổng quan, sinh viên sử dụng biểu đồ gói UML (UML Package Diagram) nhằm mô tả cấu trúc phân tầng và mối quan hệ phụ thuộc giữa các gói (package) trong hệ thống quản lý tài sản. Biểu đồ này cho phép làm rõ cách tổ chức các thành phần ở mức kiến trúc, đồng thời giúp đánh giá mức độ phụ thuộc giữa các module nghiệp vụ chính của hệ thống.

Hình 0.1 minh họa biểu đồ phụ thuộc gói của hệ thống. Các gói được sắp xếp theo các tầng chức năng rõ ràng, bao gồm tầng dùng chung (COMMON), tầng nghiệp vụ cốt lõi (ASSET_MANAGEMENT và USER_MANAGEMENT) và các

tầng hỗ trợ, khai thác dữ liệu (AUDIT_TRACKING và REPORTING). Việc phân tầng này tuân thủ chặt chẽ các nguyên tắc thiết kế kiến trúc phần mềm, trong đó các gói ở tầng trên không được phụ thuộc trực tiếp vào các gói ở tầng dưới, các gói cùng tầng không phụ thuộc lẫn nhau, và không tồn tại sự phụ thuộc bỏ qua tầng.

Gói COMMON đóng vai trò là tầng nền tảng của toàn bộ hệ thống, cung cấp các thành phần dùng chung như các lớp cơ sở và hạ tầng kỹ thuật. Các gói nghiệp vụ ASSET_MANAGEMENT và USER_MANAGEMENT thiết lập quan hệ phụ thuộc kiểu «import» tới gói COMMON, cho phép kế thừa và tái sử dụng các thành phần chung mà không làm phát sinh sự phụ thuộc ngược chiều.

Các gói AUDIT_TRACKING và REPORTING không trực tiếp tham gia xử lý nghiệp vụ cốt lõi mà chỉ truy cập dữ liệu từ các gói nghiệp vụ thông qua quan hệ «access». Cách thiết kế này giúp đảm bảo tính độc lập của các module hỗ trợ, đồng thời giảm thiểu ảnh hưởng khi các gói nghiệp vụ thay đổi trong tương lai. Nhờ đó, kiến trúc tổng thể của hệ thống đạt được sự cân bằng giữa tính chặt chẽ, khả năng mở rộng và khả năng bảo trì lâu dài.



Hình 0.1: Biểu đồ phụ thuộc gói của hệ thống

0.1.3 Thiết kế chi tiết gói

Hình 0.2 trình bày thiết kế chi tiết gói của hệ thống quản lý tài sản Snipe-IT. Ở mức độ này, mỗi gói không chỉ được xem như một đơn vị kiến trúc trừu tượng mà còn bao gồm các lớp cụ thể, phản ánh trực tiếp các thực thể và dịch vụ nghiệp vụ của hệ thống.

Trong gói COMMON, các lớp cơ sở như *BaseEntity* và *BaseRepository* được định nghĩa nhằm chuẩn hóa cấu trúc dữ liệu và cơ chế truy cập dữ liệu cho toàn bộ hệ thống. Các lớp này chứa các thuộc tính và hành vi chung, đóng vai trò là nền tảng cho các thực thể nghiệp vụ ở các gói khác kế thừa, từ đó giảm thiểu sự trùng lặp mã nguồn và đảm bảo tính nhất quán trong thiết kế.

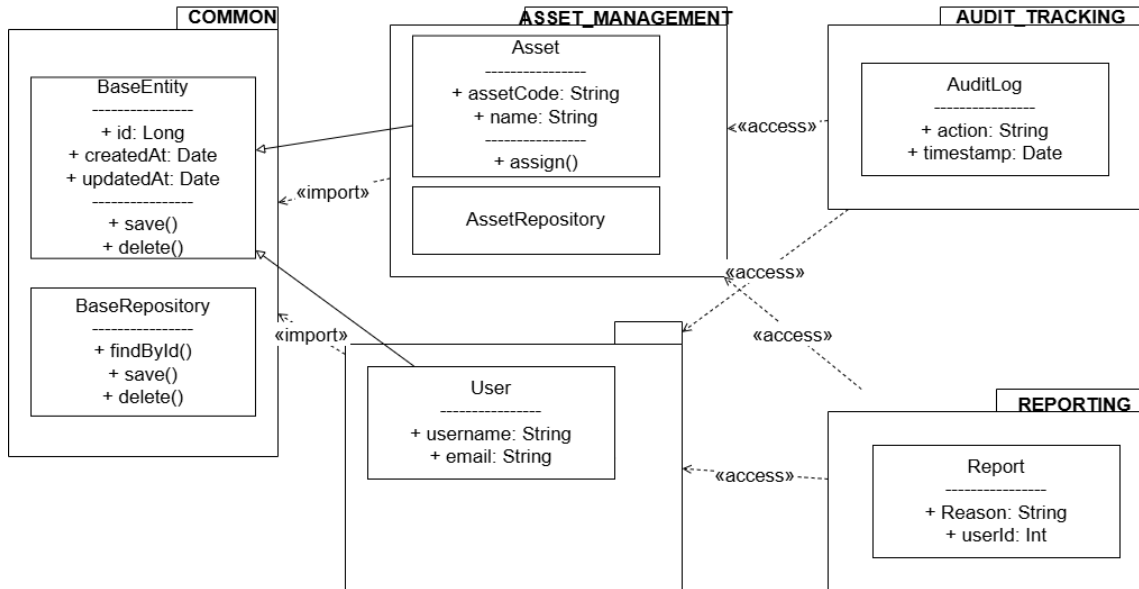
Gói ASSET_MANAGEMENT tập trung vào quản lý các nghiệp vụ liên quan đến tài sản, bao gồm các lớp mô tả thực thể tài sản, phân loại tài sản và các lớp xử lý logic nghiệp vụ tương ứng. Các lớp trong gói này kế thừa từ các lớp cơ sở trong gói COMMON thông qua quan hệ kế thừa (generalization), đảm bảo tuân thủ nguyên lý thiết kế hướng đối tượng và tạo điều kiện thuận lợi cho việc mở rộng chức năng quản lý tài sản trong tương lai.

Tương tự, gói USER_MANAGEMENT chịu trách nhiệm quản lý người dùng, vai trò và quyền hạn trong hệ thống. Các lớp người dùng được thiết kế kế thừa từ các lớp nền tảng, đồng thời thiết lập các quan hệ kết hợp (association) với các thành phần nghiệp vụ khác khi cần thiết. Cách tổ chức này giúp cô lập nghiệp vụ quản lý người dùng, hạn chế sự phụ thuộc chéo giữa các module.

Gói AUDIT_TRACKING được thiết kế nhằm theo dõi và ghi nhận các hành động quan trọng phát sinh trong hệ thống. Các lớp trong gói này chỉ truy cập thông tin từ các gói ASSET_MANAGEMENT và USER_MANAGEMENT thông qua các quan hệ phụ thuộc, không trực tiếp tham gia vào xử lý nghiệp vụ chính. Điều này đảm bảo rằng chức năng kiểm toán có thể được mở rộng hoặc thay đổi mà không ảnh hưởng đến các module cốt lõi.

Gói REPORTING đảm nhiệm chức năng tổng hợp, thống kê và xuất báo cáo dựa trên dữ liệu nghiệp vụ. Các lớp trong gói này sử dụng dữ liệu từ các gói quản lý tài sản và người dùng thông qua các quan hệ truy cập, đồng thời không làm phát sinh sự phụ thuộc ngược chiều. Thiết kế này giúp hệ thống dễ dàng bổ sung thêm các loại báo cáo mới mà không cần thay đổi cấu trúc nghiệp vụ hiện có.

Tổng thể, thiết kế chi tiết gói thể hiện rõ các quan hệ kế thừa, kết hợp, kết tập và phụ thuộc giữa các lớp và các gói, đảm bảo tuân thủ các nguyên lý thiết kế hướng đối tượng cũng như phù hợp với kiến trúc tổng thể của hệ thống quản lý tài sản



Hình 0.2: Thiết kế chi tiết các gói của hệ thống

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Phần này có độ dài từ hai đến ba trang. Sinh viên đặc tả thông tin về màn hình mà ứng dụng của mình hướng tới, bao gồm độ phân giải màn hình, kích thước màn hình, số lượng màu sắc hỗ trợ, v.v. Tiếp đến, sinh viên đưa ra các thống nhất/chuẩn hóa của mình khi thiết kế giao diện như thiết kế nút, điều khiển, vị trí hiển thị thông điệp phản hồi, phối màu, v.v. Sau cùng sinh viên đưa ra một số hình ảnh minh họa thiết kế giao diện cho các chức năng quan trọng nhất. Lưu ý, sinh viên không nhầm lẫn giao diện thiết kế với giao diện của sản phẩm sau cùng.

0.2.2 Thiết kế lớp

Phần này có độ dài từ ba đến bốn trang. Sinh viên trình bày thiết kế chi tiết các thuộc tính và phương thức cho một số lớp chủ đạo/quan trọng nhất của ứng dụng (từ 2-4 lớp). Thiết kế chi tiết cho các lớp khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục.

Để minh họa thiết kế lớp, sinh viên thiết kế luồng truyền thông điệp giữa các đối tượng tham gia cho 2 đến 3 use case quan trọng nào đó bằng biểu đồ trình tự (hoặc biểu đồ giao tiếp).

0.2.3 Thiết kế cơ sở dữ liệu

Phần này có độ dài từ hai đến bốn trang. Sinh viên thiết kế, vẽ và giải thích biểu đồ thực thể liên kết (E-R diagram). Từ đó, sinh viên thiết kế cơ sở dữ liệu tùy theo hệ quản trị cơ sở dữ liệu mà mình sử dụng (SQL, NoSQL, Firebase, v.v.)

0.3 Xây dựng ứng dụng

0.3.1 Thư viện và công cụ sử dụng

Sinh viên liệt kê các công cụ, ngôn ngữ lập trình, API, thư viện, IDE, công cụ kiểm thử, v.v. mà mình sử dụng để phát triển ứng dụng. Mỗi công cụ phải được chỉ rõ phiên bản sử dụng. SV nên kẻ bảng mô tả tương tự như Bảng ???. Nếu có nhiều nội dung trình bày, sinh viên cần xoay ngang bảng.

Mục đích	Công cụ	Địa chỉ URL
IDE lập trình	Eclipse Oxygen a64 bit	http://www.eclipse.org/
v.v.	v.v.	v.v.

Bảng 1: Danh sách thư viện và công cụ sử dụng

0.3.2 Kết quả đạt được

Sinh viên trước tiên mô tả kết quả đạt được của mình là gì, ví dụ như các sản phẩm được đóng gói là gì, bao gồm những thành phần nào, ý nghĩa, vai trò?

Sinh viên cần thống kê các thông tin về ứng dụng của mình như: số dòng code, số lớp, số gói, dung lượng toàn bộ mã nguồn, dung lượng của từng sản phẩm đóng gói, v.v. Tương tự như phần liệt kê về công cụ sử dụng, sinh viên cũng nên dùng bảng để mô tả phần thông tin thống kê này.

0.3.3 Minh họa các chức năng chính

Sinh viên lựa chọn và đưa ra màn hình cho các chức năng chính, quan trọng, và thú vị nhất. Mỗi giao diện cần phải có lời giải thích ngắn gọn. Khi giải thích, sinh viên có thể kết hợp với các chú thích ở trong hình ảnh giao diện.

0.4 Kiểm thử

Phần này có độ dài từ hai đến ba trang. Sinh viên thiết kế các trường hợp kiểm thử cho hai đến ba chức năng quan trọng nhất. Sinh viên cần chỉ rõ các kỹ thuật kiểm thử đã sử dụng. Chi tiết các trường hợp kiểm thử khác, nếu muốn trình bày, sinh viên đưa vào phần phụ lục. Sinh viên sau cùng tổng kết về số lượng các trường hợp kiểm thử và kết quả kiểm thử. Sinh viên cần phân tích lý do nếu kết quả kiểm thử không đạt.

0.5 Triển khai

Sinh viên trình bày mô hình và/hoặc cách thức triển khai thử nghiệm/thực tế. Ứng dụng của sinh viên được triển khai trên server/thiết bị gì, cấu hình như thế nào. Kết quả triển khai thử nghiệm nếu có (số lượng người dùng, số lượng truy cập, thời gian phản hồi, phản hồi người dùng, khả năng chịu tải, các thống kê, v.v.)